

```

# =====
# Name: Srushti Kale
# PRN: 23070521067
# Practical 9: Slot Filling using RNN/LSTM for SLU
# =====

import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, TimeDistributed, Bidirectional
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

# 1. Sample Data Preparation (Spoken Language Understanding - ATIS style)
# Words/Utterances
sentences = [
    "book a flight to london",
    "find a restaurant in new york",
    "show me flights from boston to denver",
    "book a train to tokyo",
    "find hotels in san francisco"
]

# BIO (Begin, Inside, Outside) Sequence Labels / Slots for each word
tags = [
    "O O O O B-DEST",
    "O O O O B-LOC I-LOC",
    "O O O O B-DEPART O B-DEST",
    "O O O O B-DEST",
    "O O O B-LOC I-LOC"
]

# 2. Text Preprocessing & Tokenization
# Tokenize words
word_tokenizer = Tokenizer(oov_token="<OOV>")
word_tokenizer.fit_on_texts(sentences)
X_encoded = word_tokenizer.texts_to_sequences(sentences)

# Tokenize tags
tag_tokenizer = Tokenizer(filters='', lower=False)
tag_tokenizer.fit_on_texts(tags)
y_encoded = tag_tokenizer.texts_to_sequences(tags)

# Padding sequences to ensure uniform input length
MAX_LEN = max([len(seq) for seq in X_encoded])
X_padded = pad_sequences(X_encoded, maxlen=MAX_LEN, padding='post')
y_padded = pad_sequences(y_encoded, maxlen=MAX_LEN, padding='post')

# Reshape y for TimeDistributed Layer
y_padded = np.expand_dims(y_padded, -1)

# Vocabulary sizes
vocab_size = len(word_tokenizer.word_index) + 1
num_tags = len(tag_tokenizer.word_index) + 1

print(f"Vocabulary Size: {vocab_size}")
print(f"Number of Tags: {num_tags}")
print(f"Max Sequence Length: {MAX_LEN}")

# 3. Build Sequence Labeling Model (Bi-LSTM)
model = Sequential([
    Embedding(input_dim=vocab_size, output_dim=32, input_length=MAX_LEN),
    Bidirectional(LSTM(units=64, return_sequences=True)),
    TimeDistributed(Dense(num_tags, activation='softmax'))
])

```

```

])

# 4. Compile Model
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
model.summary()

# 5. Train the Model
print("\n--- Training Sequence Labeling Model ---")
history = model.fit(X_padded, y_padded, epochs=50, verbose=1)

# 6. Evaluate Prediction Accuracy (Testing on a sample)
print("\n--- Model Evaluation & Prediction ---")
test_sentence = ["book flights to new york"]
test_seq = word_tokenizer.texts_to_sequences(test_sentence)
test_padded = pad_sequences(test_seq, maxlen=MAX_LEN, padding='post')

prediction = model.predict(test_padded)
predicted_tags_indices = np.argmax(prediction, axis=-1)[0]

# Reverse dictionary mapping to get tag labels back
index_to_tag = {i: t for t, i in tag_tokenizer.word_index.items()}
index_to_tag[0] = "<PAD>" # Handle padding zeros

print(f"\nInput Sentence: {test_sentence[0]}")
print("Predicted Slots:")
words = test_sentence[0].split()
for i in range(len(words)):
    if i < len(predicted_tags_indices):
        tag = index_to_tag.get(predicted_tags_indices[i], "0")
        print(f"{words[i]:<10} -> {tag}")

```

